

Time complexity

Exercise sheet 7

Exercise 7.1. Consider the following algorithm for primality testing. What is its worst-case running time in terms of the input length when N is written in unary, binary, or decimal?

```
def is_prime(N):
    for d in range(2, N):
        if N % d == 0:
            return False
    return (N >= 2) and True
```

Solution. Let n_1 , n_2 , and n_{10} be the lengths of the unary, binary, and decimal representations of N , respectively. For $N \geq 1$, we have $n_1 = N$, $n_2 = \lfloor \log_2 N \rfloor + 1$, and $n_{10} = \lfloor \log_{10} N \rfloor + 1$. The case $N = 0$ does not affect the asymptotic bounds.

The algorithm runs in $O(N)$, assuming the arithmetical operations to be elementary. Thus, with unary input it runs in $O(n_1)$; with binary input, in $O(2^{n_2})$; and with decimal input, in $O(10^{n_{10}})$.

In reality, the modulo operation $N \% d$ takes time that depends on the representation, so the algorithm with unary input need not run in linear time. It still runs in *polynomial time*. With binary or decimal input, the worst-case running time is exponential in the input length. $\square$

Exercise 7.2. A stack contains pancakes of n distinct sizes. One move reverses the top j pancakes, for a chosen j . Give an algorithm that sorts the stack in the increasing order using at most $2n - 2$ moves.

Solution. Work from the bottom of the stack upwards. For $k = n, n - 1, \dots, 2$, find the largest pancake among the top k , at position j counting from the top. If $j = k$, do nothing. Otherwise, flip the top j pancakes to bring it to the top (unless $j = 1$), then flip the top k pancakes. The largest remaining pancake is now in position k , and later flips will not move it. After the last step the pancakes are in increasing order from top to bottom. Each value of k requires at most two moves, so the total is at most $2(n - 1)$. $\square$

Exercise 7.3. The input consists of binary integers a , $b \geq 0$ and $c > 0$. Design an algorithm that computes $a^b \bmod c$ using $O(\log(b + 1))$ modular multiplications.

Solution. Use exponentiation by squaring:

```
def mod_power(a, b, c):
    if b == 0:
        return 1 % c
    if b % 2 == 1:
        return (a * mod_power(a, b - 1, c)) % c
    x = mod_power(a, b // 2, c)
    return (x * x) % c
```

Each call uses at most one modular multiplication. An odd exponent becomes even after one call, and an even exponent is halved in the next call. Hence there are $O(\log(b+1))$ calls and modular multiplications. The recursive results are smaller than c ; each product therefore has at most twice the total input length in bits. Thus the arithmetic also takes polynomial time in the input length. $\square$

Exercise 7.4. The input consists of binary integers $N \geq 0$ and $m \geq 1$. Give an algorithm that computes $\lfloor N^{1/m} \rfloor$ in time polynomial in the lengths of the two inputs.

Solution. Pending... $\square$

Exercise 7.5. Prove that the following problem 2COLOUR is in P:

Input: graph G ;

Question: does G admit a proper colouring in two colours?

Solution. Run breadth-first search from an arbitrary vertex in each connected component. Colour a vertex according to whether its distance from the starting vertex is even or odd.

If every edge joins vertices of different colours, this is a proper two-colouring. Otherwise, some edge joins vertices u and v at distances of the same parity. Their paths from the starting vertex may overlap; remove the shared beginning, and the remaining paths together with the edge form an odd cycle. Such a cycle cannot be two-coloured. Breadth-first search and the edge checks take $O(|V| + |E|)$ time. $\square$

Exercise 7.6. Prove that the following problem UNARY_SUBSET_SUM is in P:

Input: Target $t \in \mathbb{N}$ and numbers $a_1, \dots, a_m \in \mathbb{N}$, given in *unary*.

Question: Is there a set of indices $I \subseteq \{1, \dots, m\}$ such that $\sum_{i \in I} a_i = t$?

Solution. Pending... $\square$

Exercise 7.7. For a language L , let $L^* = \{w_1 \cdots w_r \mid r \geq 0, w_i \in L\}$. Prove that $L \in P$ implies $L^* \in P$. Note that $\varepsilon \in L^*$ even if $\varepsilon \notin L$.

Solution. Pending...

□

Exercise 7.8. * A *literal* is a Boolean variable x or its negation $\neg x$. A *2CNF formula* is a conjunction of clauses, each containing at most two literals. For example,

$$(x \vee \neg y) \wedge (y \vee z) \wedge (\neg z)$$

is a 2CNF formula. Let 2SAT be the problem of deciding whether a given 2CNF formula has a satisfying assignment. Prove that $2SAT \in P$.

Solution. Pending...

□